

lab5实验报告

思考题

Thinking 5.1

Q:

如果通过 `kseg0` 读写设备，那么对于设备的写入会缓存到 `Cache` 中。这是一种错误的行为，在实际编写代码的时候这么做会引发不可预知的问题。请思考：这么做这会引发什么问题？对于不同种类的设备（如我们提到的串口设备和IDE磁盘）的操作会有差异吗？可以从缓存的性质和缓存更新的策略来考虑。

A:

只有缓存块将要被新进入的数据取代时，缓存数据才会被写入内存，因此，我们向外设写入的数据无法及时地传递，会导致发生错误。根据串口设备和IDE磁盘的读写频率差异（串口设备读写频率更高），串口设备发生错误的概率要远大于IDE磁盘。

Thinking 5.2

Q:

查找代码中的相关定义，试回答一个磁盘块中最多能存储多少个文件控制块？一个目录下最多能有多少个文件？我们的文件系统支持的单个文件最大为多大？

A:

- 16个
- 16384个
- 4,194,304B (约4MB)

Thinking 5.3

Q:

请思考，在满足磁盘块缓存的设计的前提下，我们实验使用的内核支持的最大磁盘大小是多少？

A:

根据缓存区大小，最大磁盘容量为768MB

Thinking 5.4

Q:

在本实验中，`fs/serv.h`、`user/include/fs.h` 等文件中出现了许多宏定义，试列举你认为较为重要的宏定义，同时进行解释，并描述其主要应用之处。

A:

- `BLOCK_SIZE`: 磁盘块大小，文件系统读写的基本单位

- DISKMAP / DISKMAX：定义磁盘块缓存的虚拟内存范围，用于将磁盘块映射到内存
- FTYPE_REG / FTYPE_DIR：定义文件类型，用于区分普通文件和目录

Thinking 5.5

Q:

在Lab4“系统调用与fork”的实验中我们实现了极为重要的fork函数。那么fork前后的父子进程是否会共享文件描述符和定位指针呢？请在完成上述练习的基础上编写一个程序进行验证。

A:

fork进程会完整地复制父进程的地址空间，最开始父进程和子进程共享同一个文件描述符对应的 struct Fd 结构体所在的物理页，但是一旦有一个进程通过文件操作（read\write\seek）等修改了其中的文件指针，就会触发写时复制，此时父子进程不再共享同一个文件描述符，故综合来看其实是不共享的。

Thinking 5.6

Q:

请解释File,Fd,Filefd结构体及其各个域的作用。比如各个结构体会在哪些过程中被使用，是否对应磁盘上的物理实体还是单纯的内存数据等。说明形式自定，要求简洁明了，可大致勾勒出文件系统数据结构与物理实体的对应关系与设计框架。

A:

结构体	所在位置	本质	核心域及作用	使用场景
File	磁盘（fs.img） + 内存缓存	磁盘文件控制块	f_name：文件名 f_size：文件大小 f_type：文件/目录 f_direct[10]：直接块指针 f_indirect：间接块指针	文件系统服务读取磁盘时加载到内存，用于定位文件数据块、管理目录结构
Fd	内存（用户进程）	通用文件描述符头	fd_dev：设备ID fd_offset：当前读写偏移 fd_omode：打开模式	每个打开的文件或设备（如控制台、管道）在用户态都有一个Fd，用于统一访问接口
Filefd	内存（用户进程）	文件描述符扩展	内嵌Fd + f_fileid：文件ID f_dir：指向所属目录的File 实际使用中复用Fd的fd->fd_omode等	当打开的是磁盘文件时，Fd向下转型为Filefd，以便关联具体文件的元数据（如struct File*）

Thinking 5.7

Q:

图 5.9 中有多种不同形式的箭头，请解释这些不同箭头的差别，并思考我们的操作系统是如何实现对应类型的进程间通信的。

A:

文中出现了创建消息、异步消息、同步消息。同步消息通过发送后原地等待接受实现，异步消息只管发送。

实验难点

说实话，这是我目前觉得最难的一个lab了，不仅新增的代码量太大了，光是fs.c就有800多行，且定义的结构体也多，函数之间的嵌套调用也复杂，有的甚至函数跳来跳去能跳十多个函数，而且还是不同文件中的。并且文件描述符、通用文件描述符、以及磁盘文件控制块三者的关系纠缠不清，且整个过程还涉及到磁盘块脏位，要理清该何时写回等等操作。最最让人难绷的是，这么难的一个lab，竟然就只给了我们一周的学习时间，不知有多少人是急头白脸匆匆忙忙看完然后就去上机了呢。其实时间的紧张才是本实验最大的难点。

实验感想

虽然课下的时间确实紧张，但是上机还是没怎么为难我们的(extra除外)，说实话到现在为止，我lab5的代码还是没之前几个lab研究的透，课下还是要多花时间去消化一下这个庞然大物，而不是上完机就不管它了。